

DeepSearcher

# 第一阶段学习讲义

手机阅读版 · 当前项目基础与 RAG 主线

|      |                                                 |
|------|-------------------------------------------------|
| 学习范围 | RAG 入库、向量检索、在线路由、DeepSearch、ChainOfRAG、NaiveRAG |
| 学习方式 | 第一遍直接理解，第二遍模拟面试                                 |
| 生成日期 | 2026-08-18                                      |

适合手机竖屏阅读；目录与 PDF 书签可直接跳转章节。

# 目录

|                            |    |
|----------------------------|----|
| 1. 先用一分钟认识项目               | 3  |
| 2. 文档为什么要切成 Chunk          | 8  |
| 3. Embedding：把文字变成可比较的语义坐标 | 14 |
| 4. Milvus：保存并检索文档向量        | 19 |
| 5. 在线查询：先决定怎么查，再决定去哪里查     | 25 |
| 6. DeepSearch：并行研究多个方面     | 34 |
| 7. ChainOfRAG：沿着事实线索逐步追查   | 41 |
| 8. 三种查询策略总对比               | 51 |
| 9. 第一阶段不能说错的事实             | 52 |
| 10. 两轮学习方法                 | 55 |

范围：项目总览、文档切分、Embedding、Milvus、在线查询、NaiveRAG、DeepSearch、ChainOfRAG。用途：复习第一遍对话讲解。这里先讲人话和因果关系，源码名只用于需要时定位证据。配套材料：第一阶段面试问答复习手册·项目问题与优化清单 状态更新（2026-08-18）：本册保留 RAG 基础主线；PDF-only、文件级引用等描述已更新为历史学习示例。当前多格式入库、检索前授权和目录同步的进阶内容见第三阶段学习讲义。

## 1. 先用一分钟认识项目

DeepSearcher 是一个基于 RAG 的文档问答项目，解决的是：通用大模型不了解用户私有文档，回答也缺少可核验依据。

用户上传资料后，系统先提取或识别文字，把长内容切成片段，再把每个片段转换成向量并写入 Milvus。用户提问时，系统把问题也转换成向量，从 Milvus 找到相关片段，最后由当前配置的 LLM 根据这些片段生成答案；当前默认模型是 `deepseek-v4-flash`，不是固定的千问。

离线入库：资料 → 解析/识别文字 → 切分片段 → 生成向量 → 写入 Milvus

在线查询：问题 → 选择查询策略 → 选择知识库 → 向量检索 → LLM 回答

当前项目已经有受控多格式入库、检索和带引用回答，也有中文工作台、执行 Trace、Workspace/KB 访问控制、审计与独立 Worker。但“链路能运行”不代表已经达到生产标准：第二个协作数据源、ACL 撤销同步、分布式流量治理、外部身份提供商与正式 SLO 仍待完成。

必须先分清两个名称：

- **DeepSearcher**：整个项目。
- **DeepSearch**：项目中的一种多轮查询策略。

## 1.1 用户真正使用时发生什么

可以把系统想成一个“先整理资料、再开卷答题”的助手。

第一次使用时，用户上传一份 PDF。系统不会立刻让大模型阅读整份文件，而是先做资料整理：

读取 PDF 中的文字

- 把长文本切成多个可检索片段
- 为每个片段生成一串数字向量
- 把向量、片段原文和来源一起保存

之后用户提问，系统才进入开卷答题：

理解用户问题

- 决定采用哪种查询策略
- 决定应该查询哪个知识库
- 找出与问题最接近的文档片段
- 把片段作为参考资料交给当前配置的 LLM
- 生成最终回答

例如用户上传一份项目文档，其中写着：

Project Aurora 的负责人是 Alice，批准上线日期是 2026 年 6 月 15 日。

之后提问：

Project Aurora 由谁负责，什么时候批准上线？

理想链路不是让 LLM 凭记忆猜，而是先命中包含 Alice 和日期的 Chunk，再根据该证据组织答案。

## 1.2 每一层分别解决什么问题

| 层次              | 人话职责                      | 如果这一层失败            |
|-----------------|---------------------------|--------------------|
| 中文前端            | 上传文档、输入问题、观察流程            | 页面不可用或无法提交请求       |
| 本地代理            | 文件校验、临时文件、协议适配、错误映射       | 上传失败、后端错误无法友好展示    |
| FastAPI         | 接收入库与查询请求，调用 DeepSearcher | 页面可打开，但业务请求失败      |
| Loader/splitter | 提取文本并切成 Chunk             | 知识没有正确进入后续流程       |
| Embedding       | 把文本变成语义向量                 | 文档和问题无法做向量比较       |
| Milvus          | 保存并检索片段向量                 | 无法入库或无法找到证据        |
| LLM Provider    | 路由、判断、总结和生成答案             | 路由或回答失败、成本和延迟升高    |
| Trace           | 记录应用执行事件                  | 出错时只能看到最终结果，难以定位阶段 |

理解这张表的目的是不是背类名，而是遇到问题时能逐层定位。例如“页面没有答案”至少有三种完全不同的含义：

后端请求失败  
Milvus 检索失败  
检索成功但没有相关证据

它们不能用同一个“没有答案”提示掩盖。

## 1.3 本项目已经验证了什么

第一阶段历史上实际完成过：

- 真实 PDF 解析、切分、向量化和 Milvus 写入。
- 使用新入库文档完成查询。
- 使用 Project Aurora 文档回答多事实问题。
- 观察 Agent、检索文档、延迟和 Token。
- 关闭 FastAPI 验证前端 503 错误。
- 关闭 Milvus 观察向量库不可用状态。
- 使用临时 2 维 Collection 验证 L2 排序与维度不匹配。

这些证据说明当时本地功能链路可用，但只能证明“在当时环境和样例中运行成功”。当前项目的后续验证应以 [docs/开发记录/验证记录/](#) 中的 2026-08-16 至 2026-08-17 记录为准；它们仍不能证明：

- 高并发稳定性。
- 生产可用 SLA。
- 检索召回率达到目标。
- 回答准确率达到目标。

- 多租户数据隔离已经完成。

第一遍学习必须建立这种边界意识：能演示什么，就说什么；没有测过的，不用“应该没问题”代替证据。

## 2. 文档为什么要切成 Chunk

RAG 需要召回与问题最相关的局部证据。如果整篇 PDF 只生成一个向量，用户问一个细节时，系统也只能召回整篇文档：检索颗粒度粗，而且大量无关内容会进入大模型上下文。

项目先通过 Loader 提取文本，再通过 splitter 切成 Chunk。Chunk 才是实际的向量化和检索单元。

### 2.1 Chunk 大小的取舍

Chunk 太大：

- 上下文相对完整。
- 检索颗粒度较粗。
- 命中片段可能包含较多无关内容。
- 单个片段占用更多 Token。

Chunk 太小：

- 检索颗粒度更细。
- 单个片段无关内容可能更少。
- 语义容易被切断。
- Chunk、Embedding 和向量存储数量增加。



- 为补足上下文而提高 top-k 后，总 Token 不一定下降。

因此不能说“Chunk 越小越精准”，更准确的是：Chunk 变小通常会提高检索颗粒度，但最终召回质量必须通过评测验证。

## 2.2 overlap 与 wider\_text

**chunk\_overlap** 让相邻 Chunk 保留少量重复文字，主要缓解关键信息恰好位于切分边界的问题。

overlap 不能无限增大，否则会导致：

- Chunk 重复度提高
- 重复向量和存储增加
  - 检索可能返回多个近似片段
  - 占用 top-k 名额
  - 重复上下文浪费 Token

**wider\_text** 是围绕命中 Chunk 保存的一段更宽上下文，主要在回答生成阶段使用。

两者的区别：

| 概念         | 发生阶段     | 主要作用       |
|------------|----------|------------|
| overlap    | 文档切分     | 缓解切分边界断裂   |
| wider_text | 命中后的回答生成 | 补充局部片段的上下文 |

项目采用的是“短文本检索、宽上下文生成”思路：较短 Chunk 保持检索颗粒度，命中后再用 `wider_text` 帮助模型理解完整语境。

## 2.3 当前参数不能说成最优

当前默认参数是 `chunk_size=1500`、`chunk_overlap=100`。同一份 `What is Milvus.pdf` 的本地实验中，`1500/100` 产生 10 个 Chunk，改成 `500/50` 后产生 25 个 Chunk。

这个实验只证明参数会改变片段数量，不能证明默认值最优。正确评测方法是准备带标准证据片段的问题集，对不同参数比较：

- Recall@K。
- 回答正确率和忠实度。
- 重复片段比例。
- 入库时间与存储量。
- 查询延迟和 Token。

## 2.4 用一个具体例子理解切分边界

假设原文是：

Project Aurora 的负责人是 Alice。该项目经过安全评审后，  
于 2026 年 6 月 15 日批准上线。

如果刚好在中间切开：

Chunk A: Project Aurora 的负责人是 Alice。该项目经过安全评审后

Chunk B: 于 2026 年 6 月 15 日批准上线。

用户问“谁负责、什么时候上线”时，单独命中任一 Chunk 都不完整。overlap 会让边界附近的文字同时出现在相邻片段中，降低两项事实被完全拆散的概率。

但如果 overlap 接近 Chunk 本身长度：

Chunk A: 正文 500 字

Chunk B: 其中 450 字与 A 重复

系统会为大量重复文字生成向量。查询时 A、B 可能同时进入 top-k，看似召回了两条，实际上只有一份证据；最终 Prompt 还会重复消费 Token。

## 2.5 为什么小 Chunk 不一定降低总 Token

一个常见误区是：

Chunk 从 1500 改成 500，单片段更短，所以总成本一定下降。

单片段 Token 的确下降，但系统可能需要召回更多片段才能恢复完整语义。例如：

方案 A: top\_k=3，每片 1500 字符

方案 B: top\_k=8，每片 500 字符

方案 B 还会产生更多 Embedding、更多向量存储和更多结果去重工作。因此最终成本需要同时计算：

- 入库阶段的 Chunk 数和 Embedding 数。
- Milvus 存储与索引规模。
- 查询阶段实际返回多少片段。
- `wider_text` 最终放入多少上下文。
- 大模型实际消耗多少 Token。

## 2.6 PDF 解析与切分不是一回事

解析回答的是“文件里读出了什么”，切分回答的是“把读出的文本怎样分段”。

第一阶段以 PDF 说明解析与切分。当前 LoaderRegistry 还支持 Excel、PPTX 与图片 OCR；但 PDF 文本层解析本身仍更适合排版简单的文件。以下内容可能出现问题：

- 扫描 PDF：页面只有图片，需要 OCR。
- 多栏排版：文字提取顺序可能错乱。
- 复杂表格：单元格关系可能丢失。
- 图片与公式：纯文本抽取无法保留语义。
- 页眉页脚：可能反复进入每个页面文本。

即使 splitter 工作完全正确，如果 Loader 一开始就读错了内容，后续 Embedding 和检索也只能在错误文本上工作。

## 2.7 当前引用能定位到哪里，仍缺什么

“只能到文件级”是早期历史结论，已不适用于当前工作树。现有 Citation、Evidence 与 Trust Layer 会保留来源和可审计的内部定位，PDF 也有页码/位置相关验证；文档业务时间、版本系列和最终 Evidence 快照进一步参与一致性与谱系判断。

这不等于“引用已证明答案正确”：Citation Span 证明模型看到的证据位置可定位，确定性一致性和可选 Entailment 才分别检查数字/日期/条件与语义支持。扫描件、多栏排版、复杂版面和 OCR 的质量仍可能限制定位，因此应报告 Citation 状态和验证结果，不能只因有链接或页码就宣称完全 grounded。

## 2.8 这一部分的标准表达

项目不会直接把整篇文件生成一个向量，而是通过 Loader 提取或识别结构化内容，再由 splitter 形成 Chunk；Chunk 才是实际的向量化和检索单元。较小 Chunk 可以提高检索颗粒度，但过小会割裂语义并增加向量数量，所以项目使用 overlap 缓解边界断裂，并在生成时优先使用 wider\_text 补充上下文。表格与代码还要先保护原子结构，vector\_text 只服务 embedding，不能替代展示与引用文本。当前 1500/100 仍是默认参数，不可宣称最优。

## 3. Embedding：把文字变成可比较的语义坐标

Embedding 把文本转换成固定长度的一组浮点数。语义相近的文本，在同一向量空间中通常更接近。

文档 Chunk → 文档向量  
用户问题 → 查询向量  
文档向量与查询向量计算距离 → 找到候选证据

Embedding 只负责语义表示和候选召回，不能保证：

- 文档事实正确。
- 正确片段一定进入 top-k。
- 大模型正确理解片段。
- 最终答案没有幻觉。

因此检索评测与生成评测必须分开。

### 3.1 为什么维度必须一致

Milvus 创建 Collection 时会固定向量字段维度，例如：

```
embedding FLOAT_VECTOR(dim=1024)
```

如果文档使用 1024 维向量入库，查询却换成 1536 维向量，Milvus 无法逐坐标计算距离，会报维度不匹配。

还要注意：两个模型即使都是 1024 维，也不代表可以混用。不同模型可能使用完全不同的语义空间，维度一致只是必要条件，不是兼容性保证。

### 3.2 L2 距离怎样解释

当前项目默认使用 L2，也就是欧氏距离。

必须记住：

L2 是距离，越小越近。

项目的 2 维临时 Collection 实验得到距离 0.01、0.81、4.81，排序从小到大。虽然结果字段名叫 `score`，当前实际保存的是 Milvus 返回的 `distance`，不能说“score 越大越相似”。

三种常见度量的人话区别：

| 度量     | 直观含义            |
|--------|-----------------|
| L2     | 比较绝对几何距离        |
| Cosine | 比较向量方向是否一致      |
| IP     | 比较内积，同时受方向和模长影响 |

选择度量应参考 Embedding 模型建议、向量是否归一化，以及真实评测结果。

### 3.3 为什么换模型通常要重新入库

切换 Embedding 模型可能改变维度、语义空间、归一化方式和推荐 metric。稳妥迁移流程是：

保留旧 Collection

- 创建新版本 Collection
- 使用新模型重新生成向量
- 运行检索回归评测
- 切换查询流量
- 保留回滚能力

不能在沒有备份和评测的情况下直接删除旧 Collection。

## 3.4 用一个二维例子理解向量

真实项目使用 1024 维向量，人脑很难直接观察。学习时可以先看二维坐标：

问题 Q: [0, 0]

文档 A: [0.1, 0]

文档 B: [0.9, 0]

文档 C: [2.1, 0]

如果使用 L2，Q 到 A、B、C 的距离分别是 0.1、0.9、2.1。A 距离最小，应排在最前。真实 Embedding 只是把二维扩展到数百或数千维，基本思路不变。

这也解释了维度错误为什么不是“小误差”。二维向量 [0.1, 0.8] 和三维向量 [0.1, 0.8, 0.3] 连数据结构都不一致，无法逐坐标计算同一种距离。



## 3.5 为什么同维模型也不能混用

可以把两个 Embedding 模型想成两张不同坐标系的地图。即使两张地图都有 1024 个坐标，模型 A 的第一个坐标也不等于模型 B 的第一个坐标含义。

因此文档向量来自模型 A、问题向量来自模型 B 时，计算出的距离没有可靠语义。知识库不仅应该记录 dimension，还应记录：

- provider 与模型名称。
- 模型版本。
- 文本预处理方式。
- 是否归一化。
- 使用的 metric。
- 文档与 Chunk 配置版本。

## 3.6 L2 与 Cosine 为什么可能给出不同直觉

考虑两个向量：

```
[1, 0, 0]  
[9, 0, 0]
```

它们方向完全一致，所以 Cosine 相似度为 1；但长度相差 8，所以 L2 距离并不小。

这说明“哪个更相关”不是脱离模型和 metric 的绝对概念。Embedding 模型在训练时通常对向量空间和

推荐度量有预期，工程上应该遵循模型说明，再用真实问题集验证。

## 3.7 如何判断错误是否来自 Embedding

如果最终答案错误，不应该马上更换模型。先把链路拆开：

1. 原文中是否真的包含标准事实？
2. Loader 是否正确提取该事实？
3. splitter 是否生成了包含完整事实的 Chunk？
4. 该 Chunk 是否进入 Milvus？
5. 使用当前问题查询时，它排在第几？
6. 改写问题、提高 top-k 或换 Embedding 后排名如何变化？

只有正确 Chunk 已经存在且切分完整，却长期无法被语义相近的问题召回，才应重点怀疑 Embedding 或检索度量。

## 3.8 这一部分的标准表达

DeepSearcher 离线阶段对每个 Chunk 生成固定维度的 Embedding，在线阶段用同一个模型把用户问题转换成查询向量，再由 Milvus 计算近邻。当前配置使用 1024 维和 L2，因此距离越小越近。维度一致只是 Schema 的必要条件，如果更换模型，即使维度相同，语义空间也可能不同，所以通常要创建新 Collection、重新向量化并通过评测后再切换。

## 4. Milvus：保存并检索文档向量

Milvus 在项目中承担向量数据库角色。Collection 可以先理解为带有固定 Schema 的“向量表”。

当前核心数据包括：

- **embedding**：Chunk 向量。
- **text**：Chunk 文本。
- **reference**：来源文件或网页。
- **metadata**：附加信息，例如 **wider\_text**。

### 4.1 入库与查询

入库时写入的是一组对应数据，而不只是向量：

向量 + 原文 + 来源 + metadata

查询时：

#### 用户问题

- 生成 query vector
- Milvus 按 metric 搜索
- 返回 top-k 候选
- 得到文本、来源、metadata 和 distance

**top\_k=5** 只表示取排名前五的候选，不表示这五条一定相关。生产系统通常还需要距离阈值、Rerank、metadata 过滤和检索评测。

## 4.2 Collection、索引和 metric 的分工

- Collection：规定存什么数据以及向量维度。
- 索引：决定如何更快地寻找近邻。
- metric：决定怎样计算“近”。

索引用构建时间、存储或近似误差换取查询速度，所以必须同时观察延迟和 Recall@K。

## 4.3 空结果不一定是没有答案

空结果可能来自：

1. 知识库确实没有相关内容。
2. 选错 Collection。
3. Collection 不存在。
4. Milvus 连接失败。
5. 向量维度不匹配。
6. Embedding 调用失败。
7. 检索参数不合理。

8. 调用方错误地把依赖异常降级为空结果。

当前 Milvus 适配层已将搜索、集合和连接异常映射为 `VectorSearchFailed` 等明确错误；因此“空结果”应表示搜索完成但没有命中，而不是依赖故障的通用替身。调用方仍必须保留错误分类，不能自行把异常吞成 []。

## 4.4 Milvus Lite 与 Docker Milvus

| 方式            | 适合阶段             | 主要限制               |
|---------------|------------------|--------------------|
| Milvus Lite   | 个人学习、原型、小数据测试    | 不能代表独立网络服务的故障和扩展能力 |
| Docker Milvus | 学习真实服务连接、资源限制和故障 | 部署和资源成本更高          |

本地已用 Docker Milvus 验证建表、插入、L2 排序和维度不匹配。当前环境缺少 `milvus_lite` 依赖，因此不能声称所有 Lite 测试均通过。

## 4.5 为什么不能滥用 `force_new_collection`

`force_new_collection=True` 可能删除已有 Collection 后重建。本地实验方便，生产环境却可能造成数据丢失。更安全的是创建版本化 Collection、验证后切流，并保留旧版本回滚。

## 4.6 一条记录在 Collection 中是什么样

可以把一条 Milvus 数据理解为：

id: 自动主键  
embedding: 用于数学检索的向量  
text: 实际 Chunk 原文  
reference: 来源文件或 URL  
metadata: wider\_text、页码等附加信息

向量只能告诉系统“哪条记录距离近”，不能直接还原答案。命中后仍需要 `text` 或 `wider_text` 交给大模型，也需要 `reference` 告诉用户证据来自哪里。

## 4.7 top-k 到底表示什么

`top_k=5` 只表示按当前 metric 返回排名最靠前的五条候选。它不表示：

- 五条都与问题相关。
- 第五条达到了合格阈值。
- 第一条一定包含正确事实。
- 没进入前五的片段一定无关。

top-k 太小可能漏掉标准证据；太大则会把更多无关或重复内容送进 Prompt。完整检索策略通常还会结合距离阈值、metadata 过滤、跨 Collection 合并、Rerank 和上下文 Token 预算。

## 4.8 索引为什么不是“建了就一定更好”

如果向量数量很少，逐个比较也能很快完成。数据量增长后，索引用额外构建时间、存储和内存换取更快搜索。

近似向量索引的典型取舍是：

- 搜索更快
- ☐ 可能漏掉少量真实近邻

因此评估索引不能只看延迟，还要同时看 Recall@K。如果把 P95 从 500 ms 降到 100 ms，却让大量标准证据掉出 top-k，这个优化对问答系统并不合格。

### 4.9 为什么选择 Milvus 没有唯一答案

| 方案                | 适合场景                        | 需要承担的代价               |
|-------------------|-----------------------------|-----------------------|
| FAISS             | 单机算法实验、小规模本地检索              | 持久化、服务化、权限和元数据管理要自行补充 |
| pgvector          | 已有 PostgreSQL，向量需与关系数据库联合查询 | 纯向量规模扩大后要评估索引和数据库压力   |
| Milvus Lite       | 个人学习、原型、小数据测试               | 不能模拟独立网络服务与集群故障       |
| Docker/服务化 Milvus | 向量检索是独立能力，数据和并发持续增长         | 需要单独部署、监控和容量治理        |

所以面试时不要只说“Milvus 性能高”。更专业的说法是：当前项目选择 Milvus，是因为它提供 Collection、向量索引、metadata 与独立客户端接口；但小规模项目未必需要承担独立服务的运维成本。

## 4.10 用故障分类理解空结果

理想接口应该把以下情况分开：

|                             |                |
|-----------------------------|----------------|
| SEARCH_SUCCESS_WITH_RESULTS | 搜索成功且有结果       |
| SEARCH_SUCCESS_NO_HIT       | 搜索成功但无命中       |
| COLLECTION_NOT_FOUND        | Collection 不存在 |
| VECTOR_DB_UNAVAILABLE       | Milvus 不可用     |
| DIMENSION_MISMATCH          | 维度不匹配          |
| EMBEDDING_FAILED            | 查询向量生成失败       |

当前 `search_data()` 的部分异常会记录日志后返回 `[]`，使多个失败原因在上层收敛成同一种表现。这不仅影响用户提示，也会污染评测：系统错误如果被计作“无答案”，Recall 与空结果率都会失真。

## 4.11 混合检索已经有代码，不等于已经证明有效

当前 Milvus 适配层存在 `hybrid=True` 路径：稠密向量检索与 BM25 稀疏检索并行，再用 RRF 合并排名。

它可能帮助两类问题：

- 语义表达不同但含义相近的问题，稠密向量有优势。
- 产品编号、专有名词、日期等精确词，BM25 可能更稳。



但不能因为源码存在就声称“混合检索效果更好”。必须在同一问题集上比较 dense、BM25 和 hybrid 的 Recall@K、延迟、存储和资源开销。

## 4.12 这一部分的标准表达

Milvus 在项目里保存 Chunk 向量、原文、来源和 metadata，并按问题向量执行 top-k 搜索。Collection 固定数据结构和维度，索引负责加速近邻搜索，metric 决定距离含义。当前默认 L2 越小越近。本地实验验证了建表、插入、距离排序和维度不匹配；项目当前主要短板是部分异常会被转换成空结果，生产上必须区分真实无命中与系统故障。

## 5. 在线查询：先决定怎么查，再决定去哪里查

在线查询不是直接把问题交给大模型：

用户问题

- 选择查询策略
- 选择 Collection
- 问题向量化
- Milvus 检索
- 证据进入 Prompt
- 当前配置的 LLM 生成答案

两个路由器不要混淆：

- **RAGRouter**：选择“使用哪个查询策略”。

- **CollectionRouter**: 选择“查询哪些知识库”。

当前默认查询入口由 **RAGRouter** 在 DeepSearch、ChainOfRAG 与 NaiveRAG 三个候选之间选择；配置未显式指定时，fallback/default agent 为 NaiveRAG。NaiveRAG 同时仍是复杂策略的成本与效果基线。

## 5.1 为什么使用 LLM 路由

LLM 能根据自然语言语义做选择，比大量关键词规则灵活。但它可能输出解释文字、非法索引、不存在的 Collection，也会增加 Token 和延迟。

生产路由需要：

结构化输出 → 类型校验 → 范围/白名单校验 → 安全  
fallback → 日志与评测

当前 RAGRouter 使用严格的一基候选编号解析和范围校验，非法输出回退到配置的 fallback；CollectionRouter 也会按实际可用集合及请求允许范围过滤模型输出，并记录拒绝与 fallback。仍需通过评测观察错误路由率，不能把校验存在等同于路由一定正确。

## 5.2 NaiveRAG：最小可用基线

#### 问题

- 选择 Collection
- 问题 Embedding
- Milvus top-k
- 去重
- 使用 wider\_text 或 text
- LLM 生成答案

NaiveRAG 只检索一次、生成一次，不做查询改写、反思和迭代补充。

优点是成本低、延迟稳定、容易测试；缺点是复杂和多跳问题可能漏掉证据。复杂 Agent 是否值得使用，必须与它比较正确率、证据覆盖、延迟、Token 和失败率。

## 5.3 如何区分检索失败与生成失败

排障顺序：

#### 系统依赖是否正常

- 路由是否正确
- 标准证据是否进入 top-k
- 模型是否忠于证据

- 正确证据未进入 top-k：主要查检索侧。
- 正确证据已进入上下文但答案仍错：主要查生成侧。
- 没有结果：先排除向量库、Collection、维度和 Embedding 故障。

## 5.4 Trace 不是模型思维链

Trace 是应用主动记录的执行事件，例如 Agent、Collection、检索文档、迭代、延迟和 Token。它能帮助观察流程，但不是模型内部不可见的推理过程。

准确说法是“执行 Trace”或“阶段日志”，不能说“展示了模型完整思维链”。

## 5.5 默认在线查询到底走哪条路

项目中存在多种查询对象，最容易说错的是把 NaiveRAG 当成默认路径。

当前默认路径可以理解为：

```
online_query.query()  
→ configuration.default_searcher  
→ RAGRouter  
→ DeepSearch / ChainOfRAG / NaiveRAG
```

NaiveRAG 既会被创建为直接基线，也在默认 RAGRouter 的候选列表中；配置未指定 `default_agent` 时，它还是安全 fallback。

为什么这个事实重要？因为三种策略的成本和行为完全不同。如果实际走了 DeepSearch，却按“一次检索一次生成”估算 Token，就会严重低估调用量。

## 5.6 用一个例子区分两个路由器

当前默认系统有三种 Agent：

DeepSearch: 适合调研和多方面问题

ChainOfRAG: 适合逐步事实和多跳问题

NaiveRAG: 适合简单事实问题与低成本基线

同时有三个知识库:

technical\_docs

product\_manual

company\_policy

用户询问“比较 Milvus 与 pgvector 的架构和适用场景”时:

RAGRouter 解决: 这个问题用哪种搜索策略?

CollectionRouter 解决: 这个问题去哪些知识库查?

前者可能选择 DeepSearch, 后者可能选择 `technical_docs`。一个回答“怎么查”, 另一个回答“去哪查”。

## 5.7 LLM 路由非法输出的具体风险

RAGRouter 的候选编号如果是:

1 → DeepSearch

2 → ChainOfRAG

3 → NaiveRAG

模型理想输出是 1 或 2。现实中可能输出:

我建议选择 2，因为这是多跳问题。

9

0

没有适合的 Agent

当前代码只接受严格的一基数字；格式非法、0、越界编号都会回退到配置的 fallback，不会利用 Python 负索引。CollectionRouter 也只保留实际存在且请求允许的集合名称；不存在或越权的名称会被拒绝并进入路由决策记录。

可靠路由必须完成：

结构化输出

- 类型校验
- 候选范围或 Collection 白名单
- 租户权限过滤
- 安全 fallback
- 记录路由结果与失败原因

## 5.8 多个 Collection 为什么会影响 top-k

假设总预算 `top_k=10`，路由器选了两个 Collection。当前基础方案可能简单平均分配：

Collection A: 5 条

Collection B: 5 条

好处是总候选数和 Prompt 大小可控。问题是 A 可能高度相关却只能拿五条，B 几乎无关却仍占五个名额。

更完整的方案可以先从各 Collection 多取候选，再统一 Rerank；也可以根据路由置信度动态分配名额。但跨 Collection 的 distance 不一定天然可比较，仍需归一化或统一重排。

## 5.9 用一个完整例子理解 NaiveRAG

用户问：

Milvus 是什么？

NaiveRAG 会：

1. 选择可能包含答案的 Collection。
2. 把问题生成 query vector。
3. 从 Milvus 取 top-k Chunk。
4. 对重复结果去重。
5. 如果开启文本窗口，优先取 `metadata["wider_text"]`。
6. 把上下文和问题放进 Prompt。
7. 调用当前配置的 LLM 生成一次答案。

它不会在生成答案后继续检查“是否漏了历史、架构或应用场景”，也不会自动改写问题再查一次。简单事实题中，这种行为反而是优点：更便宜、更快、更容易解释。

## 5.10 一套可执行的错误归因方法

假设标准答案写在 PDF 中，但系统答错：

#### #### 第一步：检查系统调用是否成功

- FastAPI 是否返回错误？
- Embedding 是否成功？
- Milvus 是否可用？
- Collection 是否真实存在？

#### #### 第二步：检查路由

- 选择了哪个 Agent？
- 选择了哪些 Collection？
- 是否经过非法输出 fallback？

#### #### 第三步：检查检索

- 标准证据 Chunk 是否已入库？
- 它是否进入 top-k？
- distance、top-k 和去重后结果如何？

#### #### 第四步：检查生成

- 标准证据是否完整进入 Prompt？
- `wider_text` 是否带入过多无关内容？
- 回答是否添加了证据中不存在的信息？

只有前面三步正常，才应该优先调整 Prompt 或生成模型。否则只改 Prompt 会掩盖真正的检索问题。

## 5.11 Trace

### 应该记录什么、不应该记录什么

有价值的 Trace 包括：



- 请求 ID 和阶段时间。
- 选择的 Agent 和 Collection。
- 生成了哪些子查询。
- 检索数量、支持文档数量和来源摘要。
- 各阶段 Token 与最终状态。

不应直接暴露：

- API Key 和 Authorization。
- 用户完整私密文档。
- 未脱敏的绝对文件路径。
- 无限制的 metadata。
- 被称作“模型真实思维”的内部推理文本。

## 5.12 这一部分的标准表达

DeepSearcher 的在线查询不会直接把问题交给大模型。默认入口由 RAGRouter 在 DeepSearch、ChainOfRAG 和 NaiveRAG 之间选择策略，具体 Agent 再由 CollectionRouter 决定查询哪些知识库；未显式配置时 NaiveRAG 是 fallback/default agent。NaiveRAG 是一次问题向量化、top-k 检索和一次生成，适合作为成本与效果 baseline。LLM 路由比较灵活，当前已具备结构化校验、范围/白名单过滤与 fallback；但仍必须以评测验证策略选择质量。

## 6.

# DeepSearch: 并行研究多个方面

DeepSearch 适合报告、调研和需要多个方面证据的问题。

### 原始问题

- 生成最多 4 个子查询
- 并发执行子查询检索
- 对每个候选 Chunk 做 YES/NO 相关性判断
- 合并去重
- 判断还缺什么证据
- 生成最多 3 个补充查询
- 达到 max\_iter 或无新查询时结束
- 汇总全部证据生成答案

## 6.1 当前所谓 Rerank 是什么

项目对每个候选 Chunk 调用 LLM，只判断 YES 或 NO。严格来说，这是逐条 LLM 二分类过滤，不是真正的打分排序式 Reranker。

如果有 4 个子查询、2 个 Collection、每个返回 10 条候选，判断调用最多可能达到  $4 \times 2 \times 10 = 80$  次。因此 DeepSearch 的 Token 和延迟可能很高。

## 6.2 反思与停止

非最后一轮时，系统把原问题、历史子查询和已有 Chunk 交给 LLM。如果仍缺证据，就生成补充查询；返回空列表时提前停止；达到 `max_iter` 时强制结束。

当 `max_iter=1` 时，第一轮就是最后一轮，代码在检索后直接退出，不会进入反思。没有反思结果不是模型返回空值，而是反思代码没有执行。

## 6.3 必须诚实说明的边界

- 多子查询使用 `asyncio.gather()` 调度，但内部仍有同步调用，不能声称全链路真正非阻塞。
- 子查询和补充查询仍依赖模型输出，但已有类型、去重、上限与 fallback 校验；格式正确不等于检索策略一定有效。
- 联网搜索已作为可选 Tavily Provider 实现；只有用户显式请求且配置 `TAVILY_API_KEY` 时才启用，未配置或失败会继续只使用知识库结果。
- 没有固定评测集证明 DeepSearch 一定优于 NaiveRAG。

## 6.4 用一个问题走完 DeepSearch

原始问题：

比较 Milvus 与 pgvector 的架构、适用场景和运维成本。

这个问题包含多个意图。如果直接生成一个向量，top-k 可能集中在“架构”，遗漏“运维成本”。

DeepSearch 首先可能生成：

```
[  
  "Milvus 的架构是什么？",  
  "pgvector 的架构是什么？",  
  "Milvus 和 pgvector 分别适合哪些场景？",  
  "两者的部署与运维成本有什么差别？"  
]
```

四个子问题分别查询知识库。第一轮结束后，如果已有证据覆盖架构与场景，但缺少 pgvector 运维信息，反思阶段可能生成：

```
[  
  "pgvector 如何部署和扩容？",  
  "使用 pgvector 需要维护哪些 PostgreSQL 组件？"  
]
```

下一轮只搜索这些缺口查询，最后将所有保留证据合并总结。

## 6.5 子查询生成为什么既有价值又有风险

价值在于把混合意图拆成更容易被 Embedding 表达的简单查询。风险包括：

- 输出不是合法列表，解析失败。
- 多个子查询语义重复。
- 子查询遗漏原问题关键约束。
- 模型把原问题改写成了另一个问题。

- 简单问题被过度拆解，徒增成本。

当前 Prompt 要求模型输出 Python 字符串列表，项目再用 `literal_eval()` 解析。生产实现更适合使用 JSON Schema，并继续校验列表长度、元素类型、空字符串、重复项和与原问题的相关性。

## 6.6 `asyncio.gather()` 到底并发了什么

多个子查询被创建为任务后，通过 `asyncio.gather()` 一起调度：



但每个任务内部仍调用普通同步方法：

```
llm.chat()  
embedding_model.embed_query()  
vector_db.search_data()
```

如果这些方法执行阻塞网络 I/O，它们可能阻塞事件循环。因此准确说法是“并发调度多个子查询”，而不是“全链路已经实现高并发异步”。

改进时应优先使用异步客户端；无法异步化的阻塞调用可以放入受限线程池，同时用 semaphore 控制外部调用并发，避免同时向模型服务发出过多请求。

## 6.7 为什么逐条 YES/NO 判断非常昂贵

向量检索只给出候选，DeepSearch 随后把每个候选 Chunk 和查询交给 LLM，让它判断是否有帮助。

假设：

4 个子查询  
2 个 Collection  
每个 Collection 返回 10 个 Chunk

最坏情况下相关性判断达到：

$4 \times 2 \times 10 = 80$  次 LLM 调用

除此之外还有：

- 生成子查询。
- 每个子查询的 Collection 路由。
- 每轮补充查询反思。
- 最终总结。

所以 DeepSearch 的成本并不是简单的“比 NaiveRAG 多调用一次”。它可能多出几十次模型调用，而且延迟会随候选数波动。

更合理的优化顺序：

1. 先减少或去重候选。
2. 使用专用 Cross-Encoder/Reranker。
3. 批量判断多个 Chunk，而不是逐条调用。
4. 简单问题直接走 NaiveRAG。

5. 设置总调用数、总 Token、总时间和并发上限。

## 6.8 去重解决什么、不能解决什么

多个子查询可能命中同一个 Chunk。

`deduplicate_results()` 可以避免完全相同或按当前规则重复的结果多次进入最终上下文。

但下面两段可能语义重复、文字不同：

Milvus 是专门用于向量检索的数据库。  
Milvus 是面向向量相似度搜索的数据系统。

普通去重不一定能识别这种近似重复。更进一步可以使用文本哈希、来源与 offset、语义聚类或 Reranker 控制证据多样性。

## 6.9 最后一轮为什么不执行反思

循环顺序是：

执行本轮检索  
→ 判断是否已经达到最后一轮  
→ 是：直接结束  
→ 否：才生成补充查询

因此 `max_iter=1` 时没有反思，是控制流决定的。即使 `max_iter=3`，第三轮结束后也不会再生成第四轮查询，因为已经达到安全上限。

这与“反思返回空列表而提前停止”是两种不同情况：

- 空列表：反思执行了，并认为无需继续。
- 最后一轮：反思根本没有执行。

## 6.10 最终答案怎样生成

全部轮次结束后，项目再次对结果去重，收集所有子查询，并优先使用每个 Chunk 的 `wider_text`。  
最终 Prompt 包含：

原始问题  
所有子查询  
所有保留证据

如果没有任何检索结果，DeepSearch 会返回没有相关信息，而不是直接让模型自由回答。这种保守行为比无证据硬答更可靠，但仍无法区分“真实无命中”和“检索系统失败”。

## 6.11 应该怎样限制一次 DeepSearch

仅有 `max_iter` 还不够。一次请求还应限制：

- 最大初始子查询数。
- 每轮最大补充查询数。
- 每个 Collection 最大候选数。
- 最大 LLM 调用次数。
- 最大 Token。
- 单个外部调用超时。
- 整体请求 deadline。



- 最大并发数和取消传播。

达到预算时，系统应返回“在当前预算内得到的部分答案和证据”，并在 Trace 中说明停止原因，而不是静默截断或无限等待。

## 6.12 这一部分的标准表达

DeepSearch 适合调研和需要多个方面证据的问题。它先把原问题拆成最多四个子查询，并发执行 Collection 路由和向量检索，再用 LLM 对候选 Chunk 做 YES/NO 相关性过滤。非最后一轮时，它会根据原问题、历史子查询和已有证据生成补充查询；没有新查询或达到 `max_iter` 时结束，最后汇总全部证据。它可能提高证据覆盖，但逐 Chunk LLM 判断会显著增加 Token 和延迟，当前也没有评测证明它一定优于 NaiveRAG。

## 7. ChainOfRAG：沿着事实线索逐步追查

ChainOfRAG 适合后一个问题依赖前一个答案的多跳事实问题。

### 原始问题

- 根据历史问答生成一个简单追问
- 选择 Collection 并检索
- 根据文档生成中间答案
- 选择支持该问答对的文档
- 保存“追问 + 中间答案”
- 可选：判断信息是否足够
- 下一轮根据已有答案继续追问
- 汇总支持文档和中间问答生成最终答案

## 7.1 中间答案的价值与风险

下一轮可以利用已经发现的事实继续查找，所以适合多跳问题。但早期中间答案一旦错误，后续追问可能建立在错误事实之上，形成上下文污染和错误传播。

改进方向包括中间答案引用、低置信度事实隔离、重复追问检测和关键事实交叉验证。

## 7.2 支持文档选择

中间答案生成后，LLM 返回支持该问答对的文档索引列表。当前实现会验证列表类型、整数类型、去重与  $0 \leq \text{index} < \text{len}(\text{results})$ ，负数、浮点数、布尔值和越界值都会被拒绝；因此不会利用 Python 负索引选择末尾文档。

正确校验必须满足：

```
0 <= index < len(results)
```

## 7.3 early stopping

只有 `early_stopping=True` 时，系统才会在每轮判断当前信息是否足够。返回 `Yes` 时提前停止，否则继续到 `max_iter`。

当前配置为 `early_stopping=true`，因此会在满足最小证据条件后执行充分性判断；当显式关闭它时，系统才会执行完整最大轮数且不产生充分性反思。这不是反思返回空值，而是反思分支根本没有运行。

## 7.4 与 CoRAG 论文的境界

项目只借鉴逐步查询改写和多轮检索思想。当前实现是基于通用 LLM Prompt 的应用层循环，没有实现论文中的模型训练、拒绝采样中间检索链和完整测试时解码策略。

准确表达：

ChainOfRAG 受 CoRAG 思想启发，但不是论文训练方法的完整复现。

## 7.5 用一个多跳问题理解“链”

假设用户问：

Project Aurora 的负责人所在团队还负责哪些项目？

这个问题不是简单列出多个方面，而是后一步依赖前一步：

第一轮：Project Aurora 的负责人是谁？

中间答案：负责人是 Alice。

第二轮：Alice 属于哪个团队？

中间答案：Alice 属于 Platform Team。

第三轮：Platform Team 还负责哪些项目？

中间答案：还负责 Project Nova。

最后把多个中间事实组合成答案。

如果一开始并行生成所有问题，第二、第三个问题中的 Alice 和 Platform Team 尚未得到，查询会比较模糊。因此 ChainOfRAG 更适合这种逐步发现实体和关系的任务。

## 7.6 每一轮完整做了什么

一轮不是只有“检索”一个动作，而是：

1. 根据原始问题和历史中间问答生成一个简单追问。
2. 使用 CollectionRouter 选择知识库。
3. 为追问生成查询向量。
4. 在各 Collection 中检索并去重。
5. 让 LLM 只根据本轮文档生成中间答案。
6. 让 LLM 选择真正支持该问答对的文档。
7. 保存“追问 + 中间答案”。
8. 如果启用 early stopping，再判断信息是否已经足够。

下一轮看到的是中间问答文本，不是模型内部思维链。

## 7.7 支持文档选择为什么仍不等于事实验证

假设本轮检索到三条文档：

文档 0: Alice 负责 Project Aurora。  
文档 1: Alice 属于 Platform Team。  
文档 2: Bob 属于 Security Team。

中间问答是：

问: Alice 属于哪个团队?  
答: Platform Team。

模型理想返回：

[1]

项目随后只把文档 1 加入支持证据。

但这里有两层模型判断：先生成中间答案，再判断哪些文档支持答案。如果第一层已经答错，第二层可能继续为错误答案挑选看似相关的文档。因此它是证据筛选，不是可靠的逻辑证明器。

## 7.8 为什么负索引是一个真实漏洞

当前过滤条件大致只检查：

```
index < len(retrieved_results)
```

假设文档数为 3，模型返回 -1：

```
-1 < 3 成立
```

而 Python 列表中的 `results[-1]` 会取最后一篇文章，不会抛错。系统可能静默选择错误证据。

正确条件必须是：

```
0 <= index < len(retrieved_results)
```

还应验证元素是整数，拒绝浮点数、字符串、重复索引和超出允许数量的列表。

## 7.9 early stopping 为什么可能过早或过晚

充分性判断只要求模型回答 Yes 或 No。

过早停止：

- 已有部分证据
- 模型误判为足够
- 返回 Yes
- 最终答案缺少关键事实

过晚停止：

- 证据已经充分
- 模型仍返回 No
- 继续无意义检索
- Token 和延迟增加

因此停止条件不应只依赖一次主观判断。可以结合：

- 是否找到新支持文档。
- 新一轮是否增加新事实。
- 追问是否与历史重复。
- 支持证据是否覆盖原问题各要点。
- 是否达到 Token、时间或轮数预算。

## 7.10 显式关闭 early stopping 的实际含义

当前配置为：

```
early_stopping = True
```

如果显式改为关闭，每轮结束后不会调用信息充分性判断，也不会记录反思结果，系统会一直执行到 `max_iter`。

它的优点是行为更确定，不会被模型过早停止；代价是即使第一轮已经找到完整答案，也会继续执行后续轮次。

所以看到“Trace 没有反思”时，要先检查开关，而不是认为模型返回内容丢失。

## 7.11 中间答案如何污染后续查询

假设第一轮错误回答：

```
Project Aurora 的负责人是 Bob。
```

下一轮可能生成：

Bob 属于哪个团队？

此后检索即使每一步都执行正确，也是在错误前提下继续搜索。这类错误传播比单轮 RAG 更难排查，因为最终答案可能看起来逻辑连贯。

降低风险的办法：

- 中间答案必须绑定支持文档。
- 没有支持文档的答案不进入可信上下文。
- 对关键实体进行第二来源验证。
- 保留原始问题，检测追问是否偏离目标。
- Trace 展示每轮问题、答案和证据，便于人工回溯。

## 7.12 最终答案是否只依赖支持文档

最终 Prompt 同时包含：

- 原始问题。
- 所有中间问题和答案。
- 累积的支持文档。

Prompt 会提醒模型中间答案不一定准确，但提醒并不能自动纠错。如果支持文档为空，当前流程仍可能根据中间上下文生成答案。因此生产上还要定义：

- 没有支持文档时是否拒答。



- 至少需要多少条证据。
- 每个关键事实能否映射到引用。
- 无答案问题怎样安全回应。

## 7.13 与 DeepSearch 的选择原则

可以用问题结构判断：

多个方面相互独立

→ 更像 DeepSearch

后一个问题依赖前一个答案

→ 更像 ChainOfRAG

简单事实

→ 优先考虑 NaiveRAG

这只是初始规则，不是效果结论。最终仍要在真实问题集上比较准确率、证据覆盖、平均轮数、P95、Token 和失败率。

## 7.14 CoRAG

### 论文与当前项目具体差在哪里

共同思想：

- 不满足于单次检索。
- 根据不断变化的状态改写查询。
- 逐步收集信息后生成最终答案。

论文还包括：

- 训练能够逐步检索和推理的模型。

- 使用拒绝采样自动构造中间检索链。
- 在测试时通过控制检索链的长度和数量扩展计算。

当前项目主要是：

- 使用通用 LLM 和 Prompt 生成追问。
- 在应用代码中控制循环。
- 没有论文训练、拒绝采样和完整解码策略。

因此简历和面试中只能说“受 CoRAG 启发”，不能说“完整复现 CoRAG 论文”。

## 7.15 这一部分的标准表达

ChainOfRAG 适合多跳事实问题。它每轮根据原始问题和历史中间问答生成一个简单追问，再做 Collection 路由、向量检索和中间答案生成；随后由 LLM 选择支持该问答对的文档。中间问答进入下一轮，形成串行检索链。当前配置启用 early stopping，会在满足最小证据条件后判断信息是否足够；关闭时才固定运行到 `max_iter`。中间答案仍可能传播错误，但文档索引已有类型和范围校验；它只是受 CoRAG 思想启发，不是论文训练方法的完整复现。

## 8. 三种查询策略总对比

**对比项：** 查询方式

**NaiveRAG：** 一次原问题检索

**DeepSearch：** 多个研究方向，可并发

**ChainOfRAG：** 每轮一个追问，串行依赖

**对比项：** 中间答案

**NaiveRAG：** 无

**DeepSearch：** 无逐轮中间答案

**ChainOfRAG：** 每轮生成中间答案

**对比项：** 后续查询依据

**NaiveRAG：** 无

**DeepSearch：** 原问题、历史子查询和 Chunk

**ChainOfRAG：** 原问题、历史中间问答

**对比项：** 候选过滤

**NaiveRAG：** 基础去重

**DeepSearch：** Chunk 级 YES/NO 过滤

**ChainOfRAG：** 选择支持问答对的文档

**对比项：** 停止方式

**NaiveRAG：** 一轮结束

**DeepSearch：** 无补充查询或达到上限

**ChainOfRAG：** 可选充分性判断或达到上限

**对比项：** 成本与延迟

**NaiveRAG：** 较低、较稳定

**DeepSearch：** 较高，调用量可能很大

**ChainOfRAG：** 随串行轮数累积

**对比项：**适用问题

**NaiveRAG：**简单事实

**DeepSearch：**报告、调研、多方面问题

**ChainOfRAG：**多跳事实和线索追查

**对比项：**主要风险

**NaiveRAG：**证据覆盖不足

**DeepSearch：**成本高、输出格式脆弱

**ChainOfRAG：**错误中间答案向后传播

## 9. 第一阶段不能说错的事实

1. DeepSearcher 是项目；DeepSearch 是一个查询策略。
2. 默认查询路由由 RAGRouter 在 DeepSearch、ChainOfRAG 和 NaiveRAG 中选择；未显式配置时 NaiveRAG 是 fallback/default agent。
3. Chunk 是实际向量化和检索单元。
4. overlap 缓解切分边界；`wider_text` 补充生成上下文。
5. 当前 `1500/100` 是默认值，没有评测证明最优。
6. 向量维度一致只是必要条件，相同维度的不同模型仍可能不兼容。
7. 当前默认 L2 是距离，越小越近。
8. Milvus 的 `score` 字段当前实际承载 distance。
9. 空结果不一定表示知识库没有答案，也可能是系统失败。
10. LLM 路由输出必须校验，不能直接相信。

11. DeepSearch 的 Rerank 更接近逐条 YES/NO 过滤。
12. `max_iter=1` 的 DeepSearch 不执行反思。
13. 当前配置启用 ChainOfRAG early stopping; 只有显式关闭时才没有充分性反思。
14. Trace 是执行事件，不是模型思维链。
15. 当前 ChainOfRAG 不是 CoRAG 论文的完整复现。

## 9.1 对话中已经出现过的易错词

| 容易说错                 | 正确说法                                               | 为什么重要                              |
|----------------------|----------------------------------------------------|------------------------------------|
| DeepSearch 是整个项目     | DeepSearcher 是项目，DeepSearch 是查询策略                  | 混淆后无法准确介绍架构                        |
| RG                   | RAG                                                | 全称是 Retrieval-Augmented Generation |
| target 成本            | Token 成本                                           | Token 是模型输入输出计费与上下文单位              |
| reader text          | <code>wider_text</code>                            | 它表示命中 Chunk 周围的更宽上下文               |
| Chunk 小所以总 Token 一定低 | 单 Chunk 更短，但总召回数和 <code>wider_text</code> 可能让总成本上升 | 需要看完整链路成本                          |
| overlap 大会产生更多无关内容   | 更准确是产生更多重复内容和重复向量                                  | 主要问题是重复度和 top-k 名额被占用              |
| score 越大越相关          | 当前 L2 distance 越小越近                                | 字段名不能替代 metric 语义                  |
| 空结果就是没有答案            | 还可能是依赖或路由失败                                        | 必须区分系统错误与业务无命中                     |

## 10. 两轮学习方法

### 10.1 第一遍：直接理解，不现场拷问

建议按下面顺序复习，不要从类名开始背：

1. 用一分钟讲清项目解决的问题和两条主链路。
2. 解释为什么切 Chunk，以及 Chunk 大小的取舍。
3. 解释 Embedding、维度和 L2。
4. 解释 Milvus 存什么、怎样查、空结果为何不可靠。
5. 解释两个路由器与 NaiveRAG。
6. 对比 DeepSearch 和 ChainOfRAG。
7. 最后再用源码名和实验数据补充证据。

第一遍由讲解者直接说明概念、例子和标准答案，不要求学习者每一小节立刻作答。目标是先建立完整知识地图，能理解“是什么、为什么、怎么做、有什么代价、当前哪里不足”。

每学完一节，只做三件事：

1. 用自己的话复述一条主链路。
2. 标记仍不理解的词，不强行背答案。
3. 对照项目问题清单，知道当前实现的真实边界。

### 10.2 第二遍：闭卷复述与连续追问

第二遍再使用第一阶段面试问答复习手册：

1. 先用一分钟介绍项目。
2. 每题先给一句话结论。
3. 再补原因、实现、取舍和项目证据。
4. 接受连续追问，不看答案纠正表达。
5. 说到尚未实现的能力时，主动给出诚实边界。

第二遍的合格标准不是逐字背诵，而是面对不同问法仍能保持同一套因果关系和事实边界。